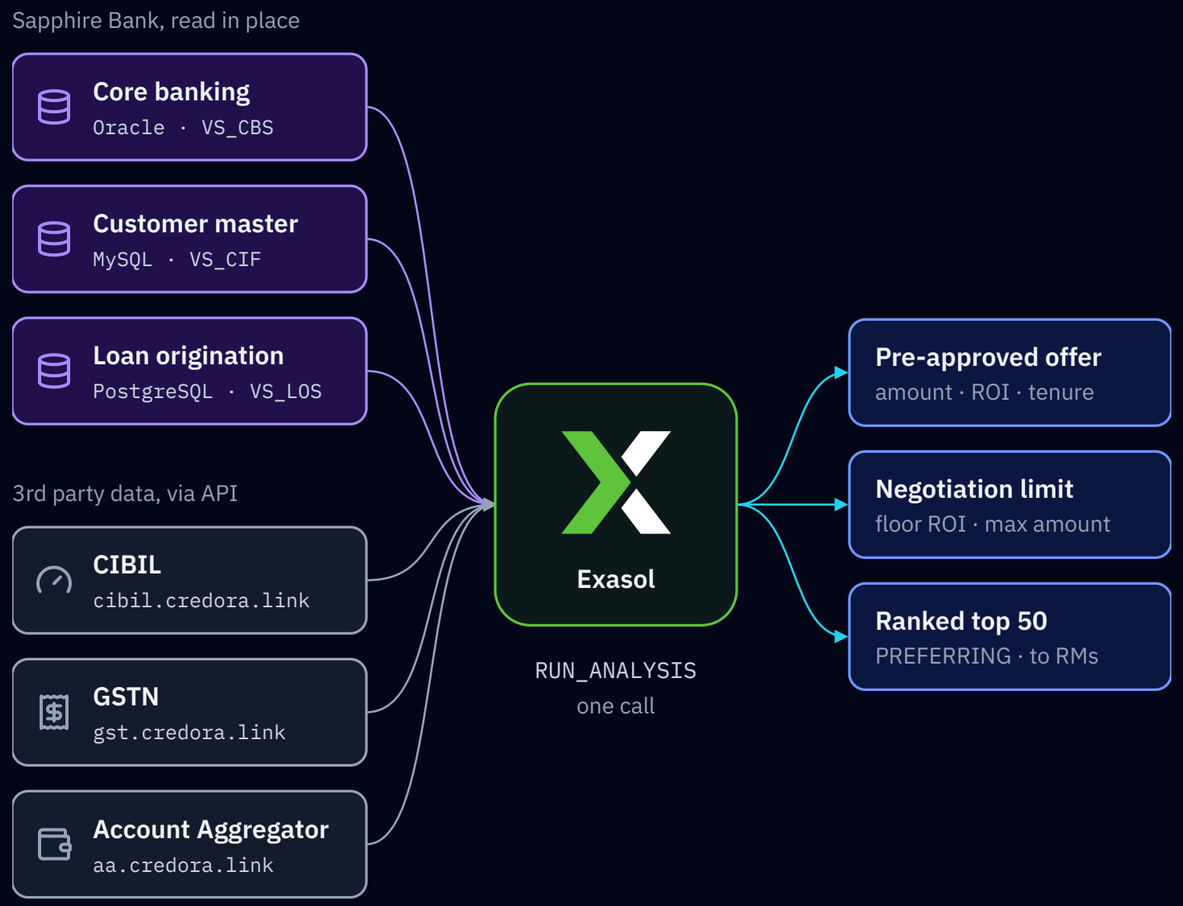
Credora
for Sapphire Bank

Pre-approved corporate credit, decided inside Exasol.

An ExaThon India 2026 project, a hands-on try-out of the Exasol Personal Local Starter Kit and Exasol Personal, deployed on AWS. Sapphire Bank is fictional and all data is synthetic.

Anamika · Product Manager @ Equitas Small Finance Bank

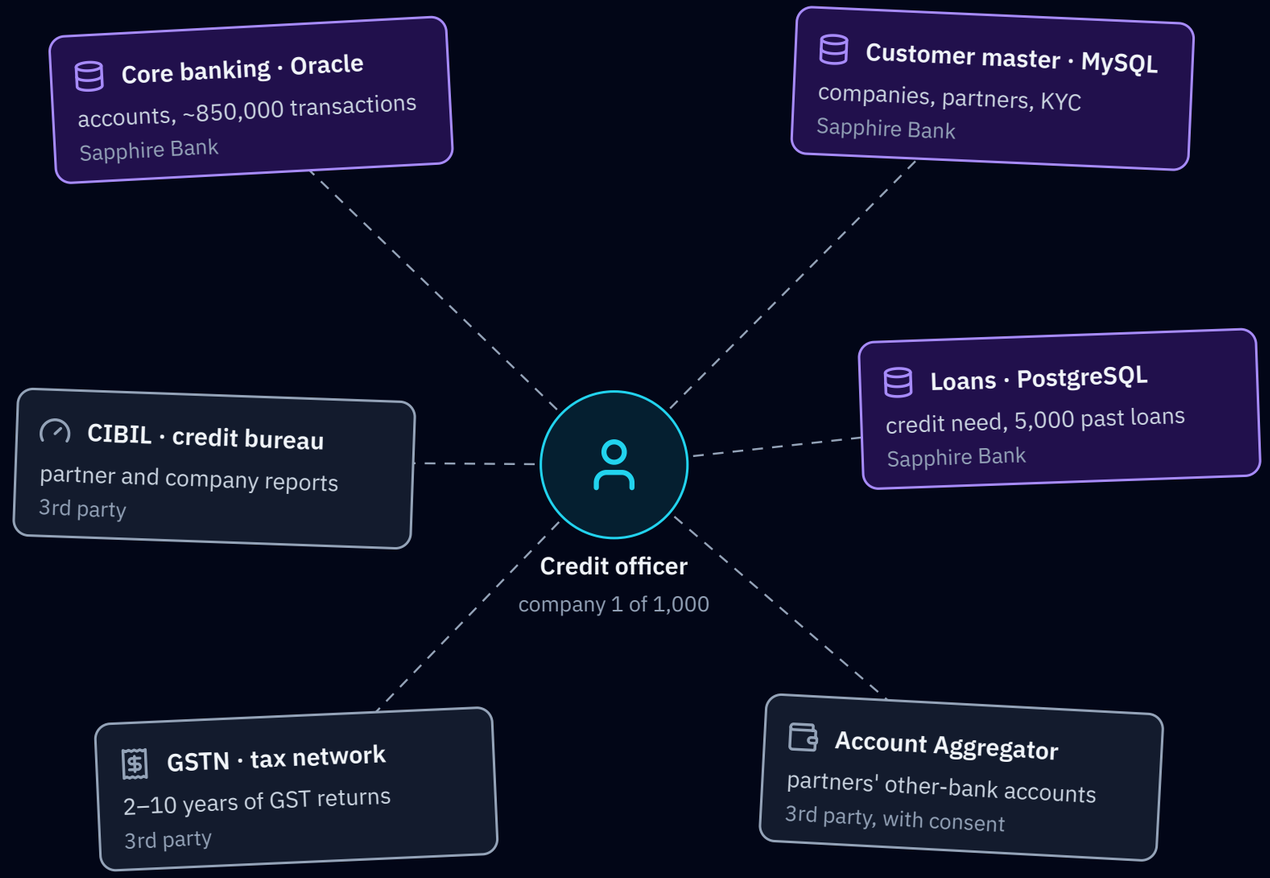
credora.loan the app **docs.credora.loan**
credora.link 3rd party data via API



Built for ExaThon India 2026 · Powered by Exasol Personal

The bank already holds the data. Offers are still slow and manual.

To make a corporate customer a credit offer, someone has to visit six systems, three in the bank and three 3rd party.



Today, one company at a time, one system at a time



With Credora, all six at once, inside Exasol

Analyse the customers we have, then offer them credit first.

Every corporate customer
existing current-account customers, 2 to 5 partners each

→

Analysed in Exasol
bank data, bureau, GST and AA data, risk model, policy

→

Pre-approved offer
amount, ROI (rate) and tenure, plus a negotiation limit

→

RM calls the customer
and negotiates inside the limit, no trip back to credit



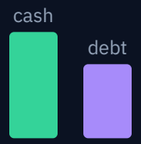
PD probability of default

How likely the company is not to repay, from a model trained on the bank's own past loans. Graded A1 (under 1%) to D (12% and above).



RAROC risk-adjusted return

(Rate – cost of funds – operating cost – PD × loss given default) ÷ capital held. Every offer must clear the 15% hurdle.



DSCR debt service coverage

Cash the company generates ÷ what the loan costs it each year. At least 1.25 for the offer, 1.10 at the negotiation limit.



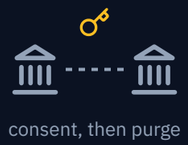
GST goods and services tax

Monthly sales each company files with the tax network, GSTN, for 2 to 10 years per company. Late filings and a gap to bank credits are warning signs.



CIBIL India's credit bureau

A score for every partner and a commercial rank (CMR) for every company. A partner below 600 or a CMR of 9 or worse means no offer.

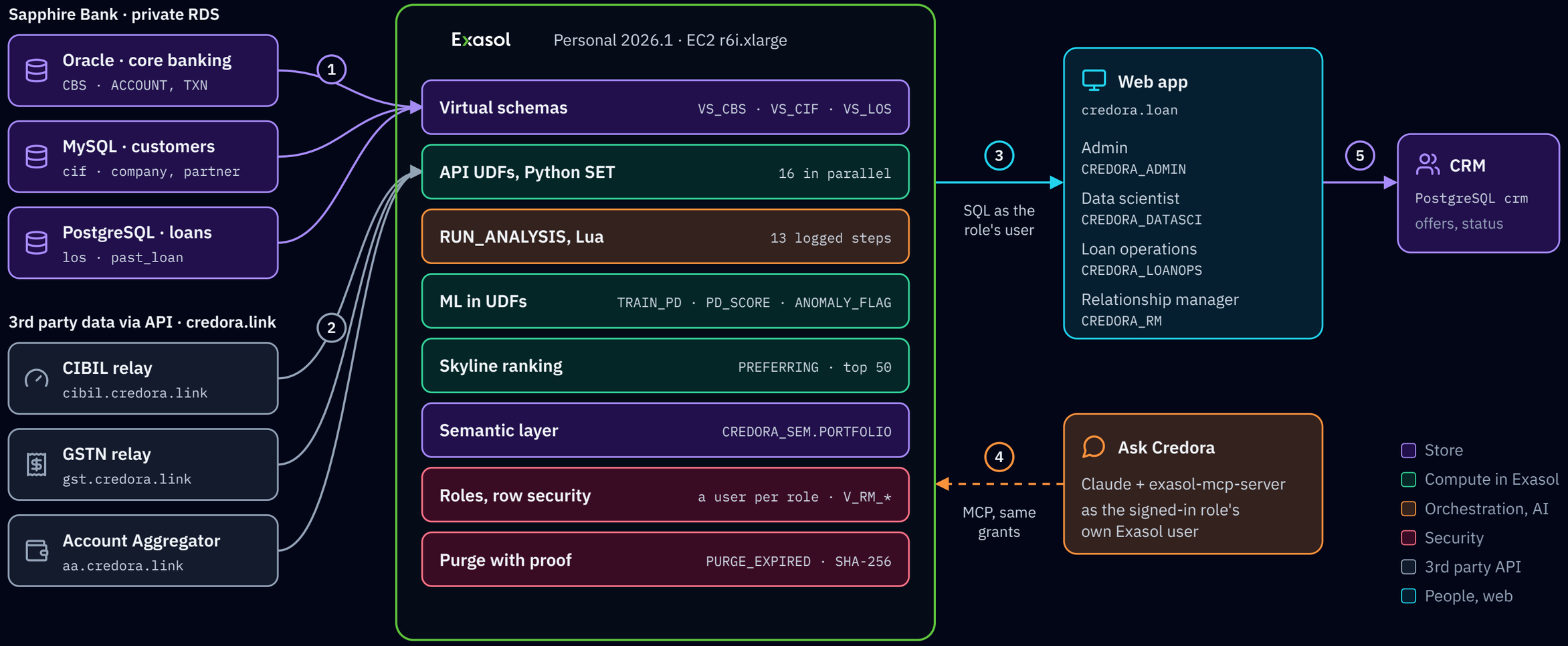


AA Account Aggregator

With the customer's consent, the partners' accounts at other banks. The consent expires, so Exasol purges the data on time, with proof.

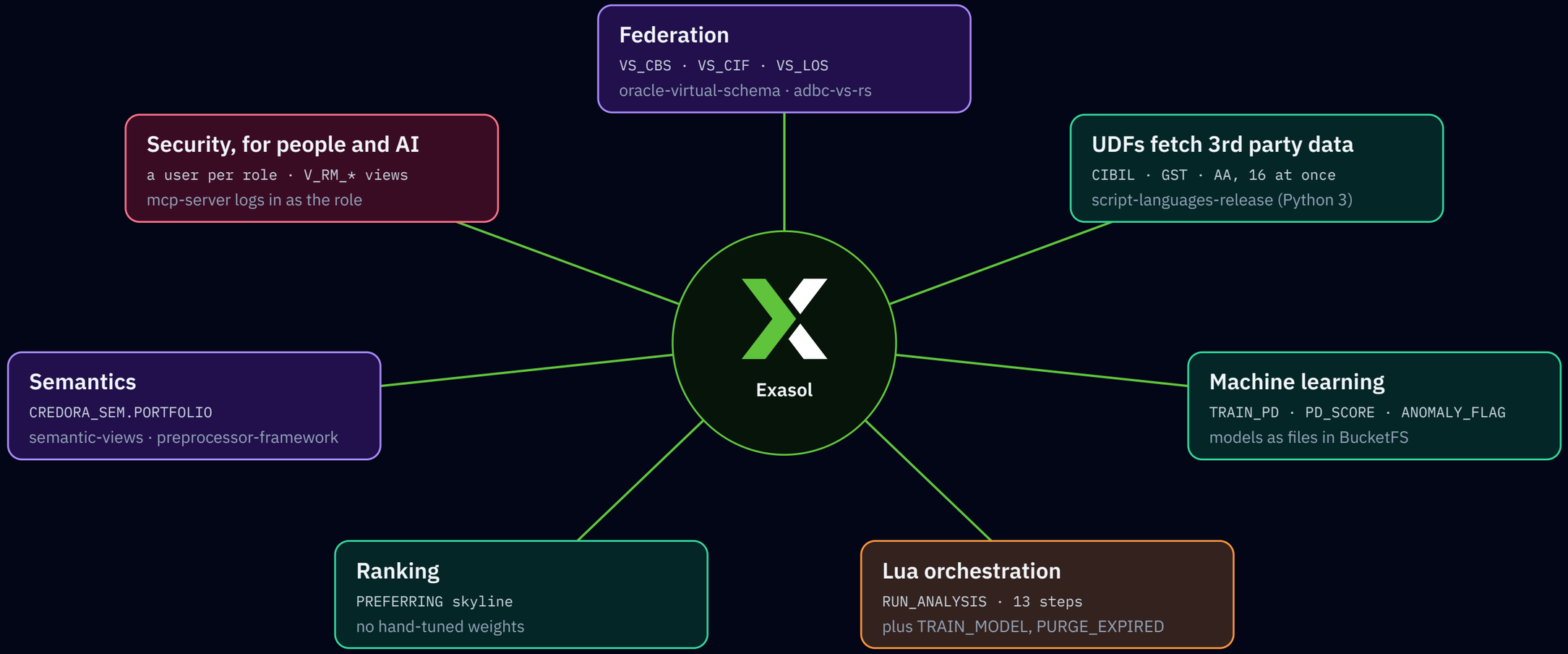
Everything meets in Exasol, and leaves as a decision.

Bank data is read in place, 3rd party data is fetched via API by UDFs, and every person and the AI read results as their own Exasol user.



One engine does seven jobs, and the data never leaves it.

Exasol Personal used to the full, with open-source extensions from both GitHub organisations, exasol and exasol-labs.

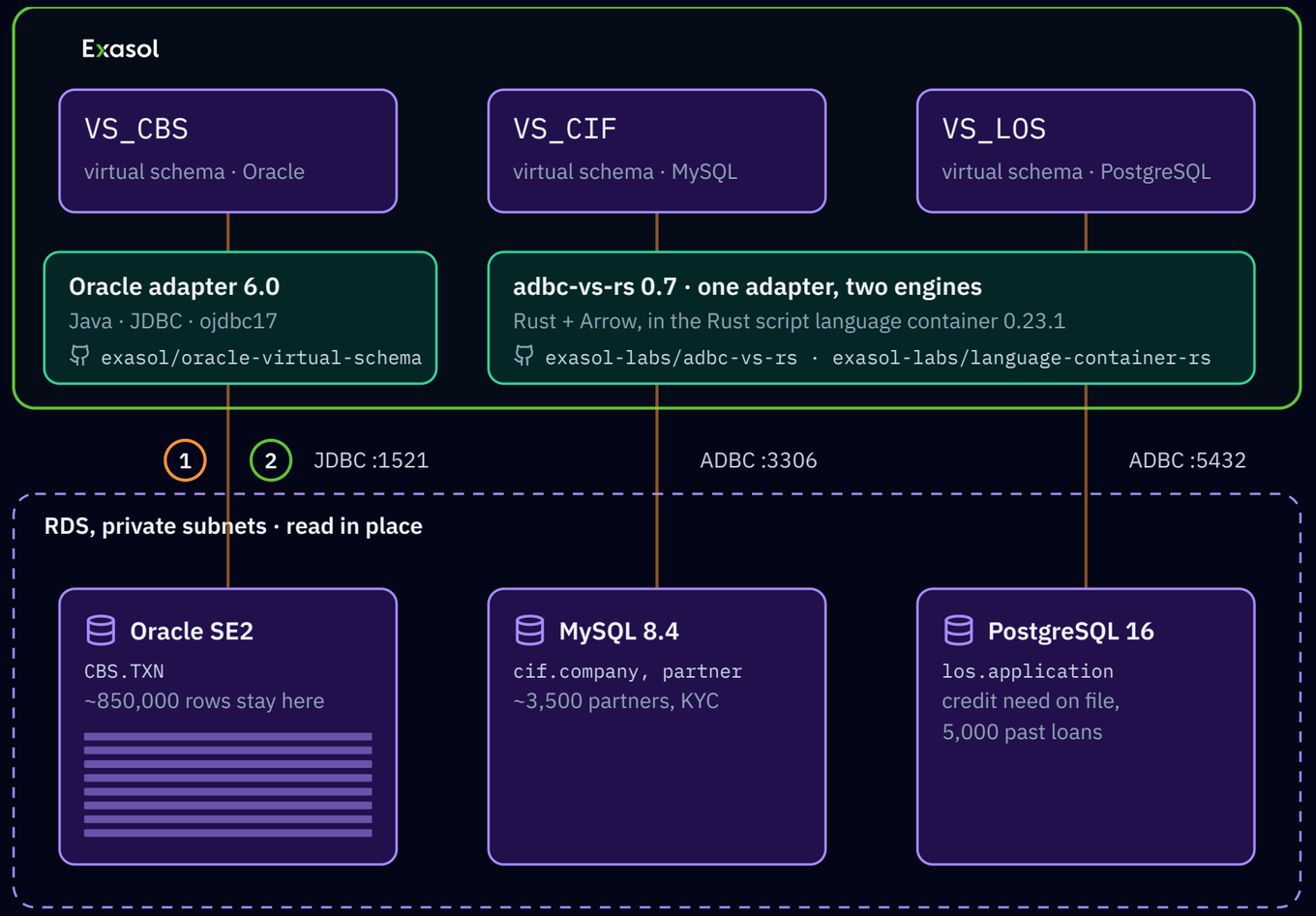


Without it you need an ETL tool, an API gateway, an ML platform, a scheduler, a ranking service, a metrics layer and an access proxy.

Virtual schemas push the work down. Nothing is copied.

Feature step 7 aggregates core banking by account and month. Exasol hands the join, the filter and the GROUP BY to Oracle.

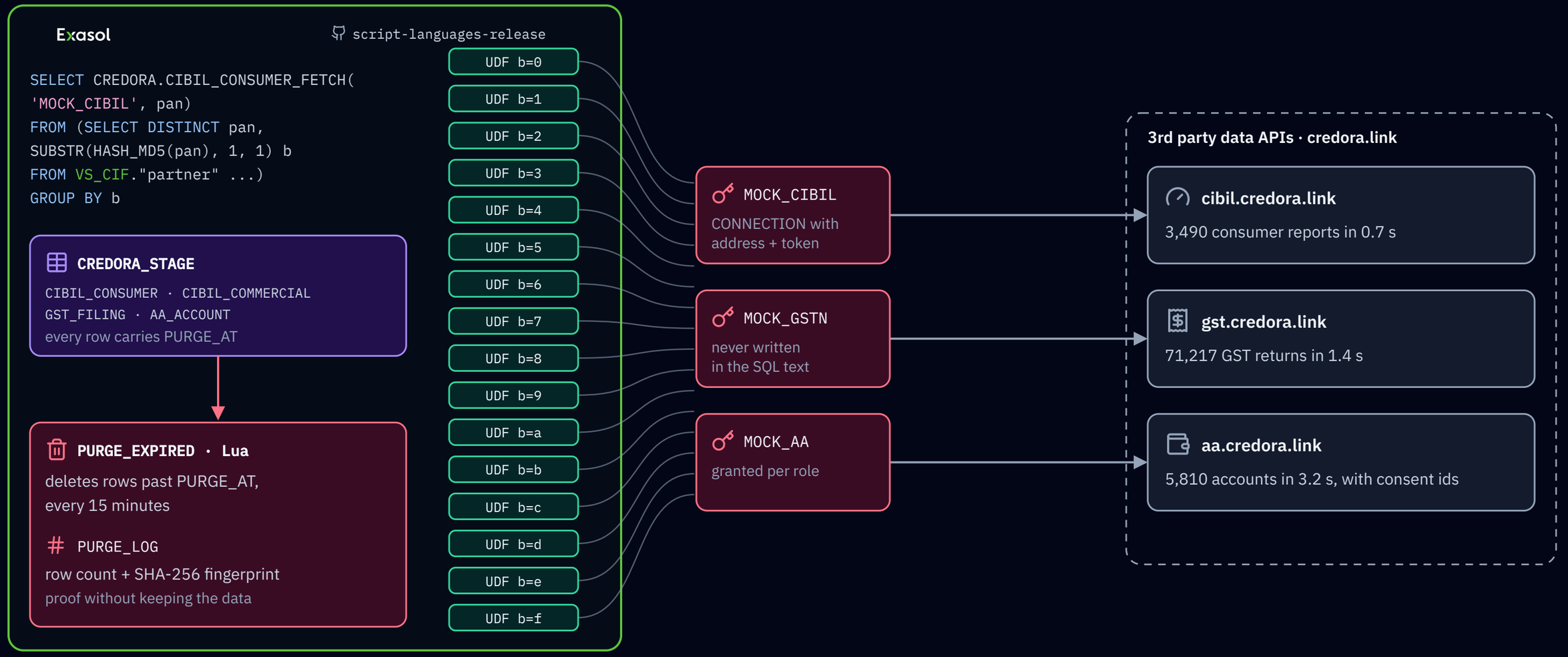
```
-- RUN_ANALYSIS, step 7
INSERT INTO CREDORA_STAGE.CA_MONTHLY
SELECT t.ACCOUNT_NO, a.OWNER_ID,
       TRUNC(t.TXN_DATE, 'MM') MON,
       SUM(CASE WHEN t.DR_CR = 'C'
           THEN t.AMOUNT ELSE 0 END),
       ...  COUNT(*) N
FROM VS_CBS.TXN t
JOIN VS_CBS.ACCOUNT a
  ON a.ACCOUNT_NO = t.ACCOUNT_NO
WHERE a.ACCT_TYPE = 'CA'
GROUP BY t.ACCOUNT_NO, a.OWNER_ID,
         TRUNC(t.TXN_DATE, 'MM')
```



Monthly totals per account come back. The ~850,000 transactions stay in Oracle, and the bank keeps one system of record.

Python UDFs fetch 3rd party data via API, 16 at a time.

A SET script groups the keys into 16 buckets by the first hex digit of an MD5 hash, and Exasol runs one UDF instance per bucket, in parallel.



One Lua call runs the whole analysis, step by logged step.

`EXECUTE SCRIPT CREDORA.RUN_ANALYSIS('<run>')` commits after every step, so the web app can watch the run as it happens.



17.5 s

Measured on the live deployment for 1,000 companies, on one r6i.xlarge node.

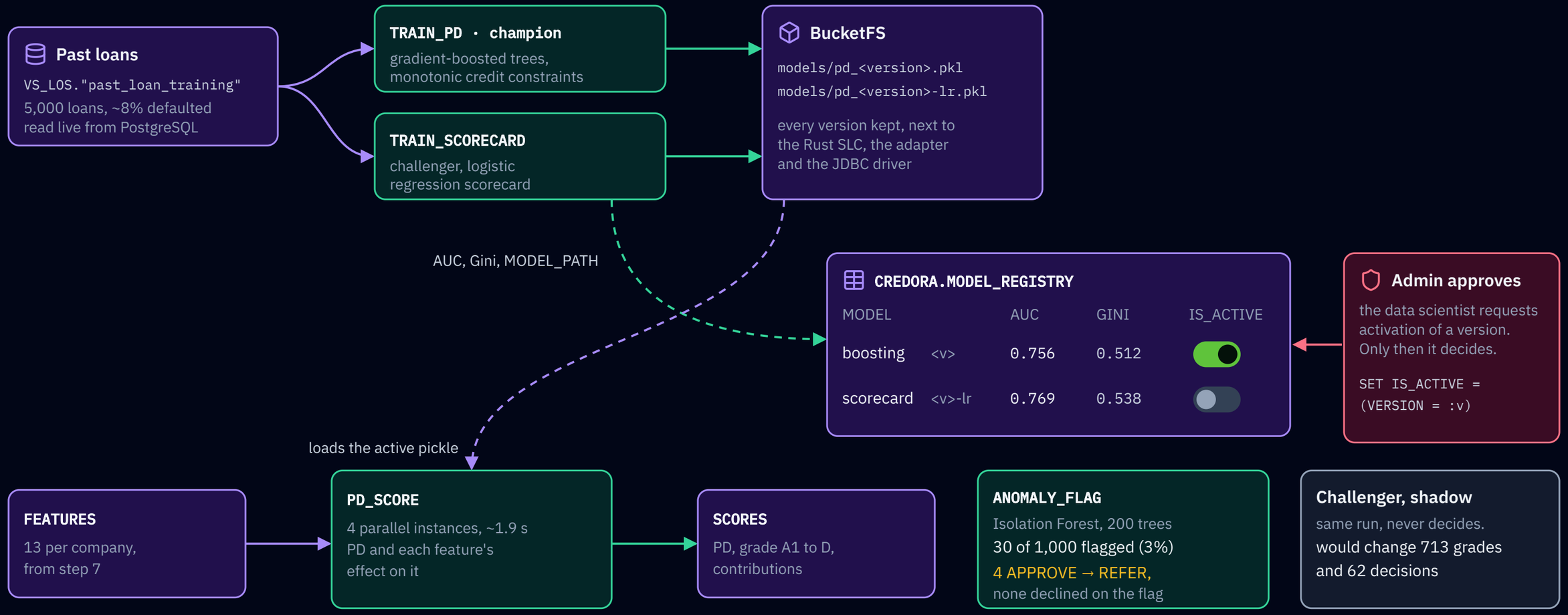
`CREDORA.RUN_LOG`

STEP	STATUS	ROWS_OUT	MS
8	OK	1	<0.1 s
9	OK	1,000	1,900
10	OK	1,000	1,300
11	OK	1,000	70
12	OK	63	30
13	OK	1,000	1,700

A failed step is logged with its error and stops the run. The last published run stays visible in the app.

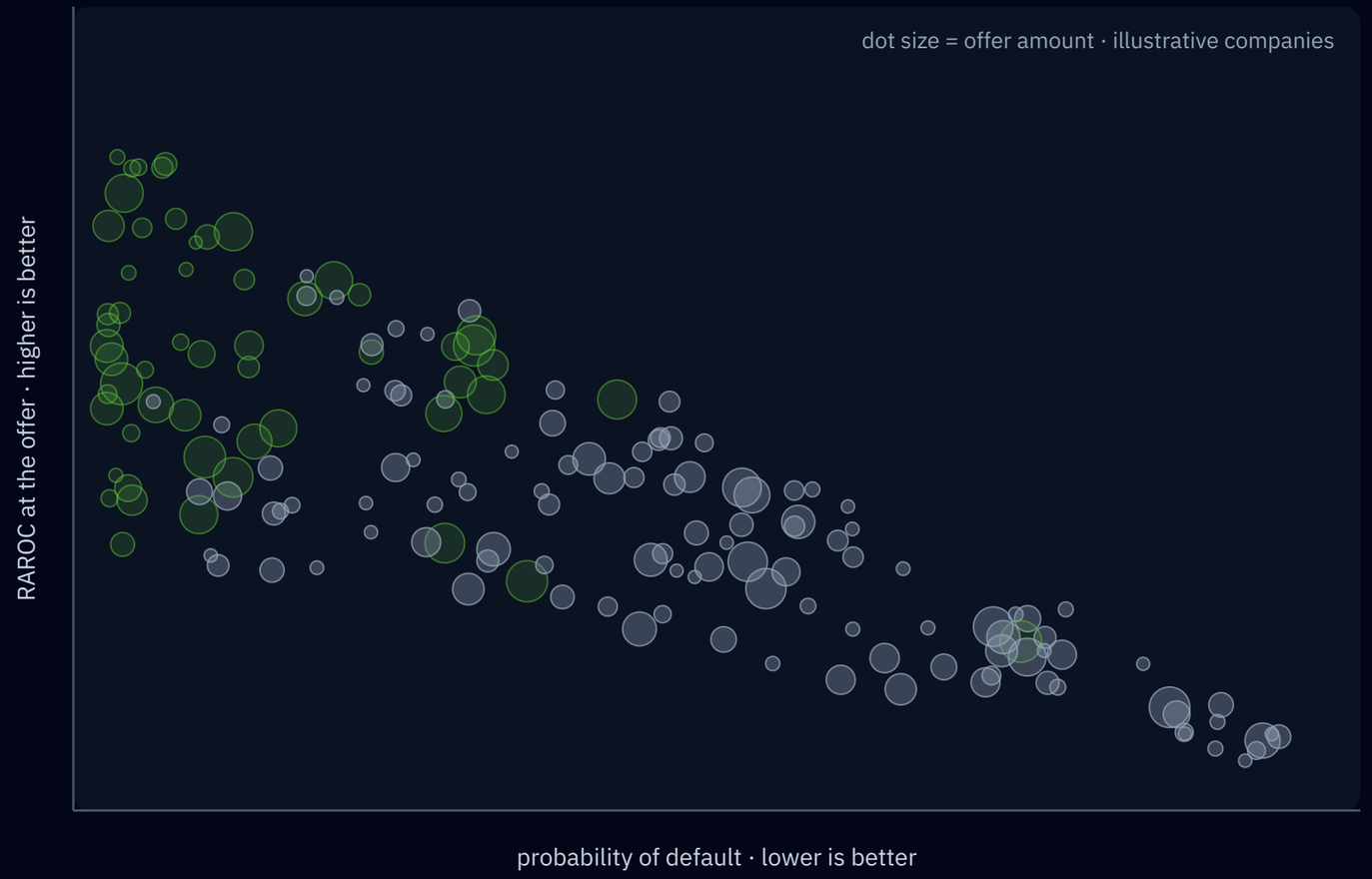
The models are trained, stored and scored inside Exasol.

Past loans are read live from PostgreSQL. Two models train on the same loans and the same hold-out. Only the active one decides.



PREFERRING ranks on three goals at once, with no weights.

The skyline is every approved company that no other beats on return, risk and amount together. Peel it off, take the next skyline, until 50 are ranked.




```
UPDATE CREDORA.DECISIONS
   SET SKYLINE_LAYER = :l
 WHERE APP_ID IN (
  SELECT APP_ID FROM CREDORA.DECISIONS
  WHERE DECISION = 'APPROVE'
    AND SKYLINE_LAYER IS NULL
  PREFERRING HIGH RAROC_OFFER
     PLUS LOW PD
     PLUS HIGH OFFER_AMOUNT)
```

56 ranked · 3 layers

50 reached: peeling stops, RANK_NO orders them

On the measured run the layers held 83 companies by the time 50 were reached, in 0.02 s. The best 50 approved go to the relationship managers.

One certified definition serves the app, the analyst and the AI.

CREDORA_SEM.PORTFOLIO holds 16 certified metrics. The SQL preprocessor compiles semantic queries per role, inside Exasol.

```
METRIC approval_rate
  AS approved_companies
     / NULLIF(companies_analysed, 0)
  ON ENTITY decision
  FORMAT 'percentage'
  DISPLAY 'Approval rate'
  SYNONYMS ('approval ratio', 'hit rate')
  RATIO PUBLIC CERTIFIED
```

model

CREDORA_SEM
.PORTFOLIO

one row per company of the latest published run

approved companies
approval rate
pre-approved amount
negotiation headroom
expected loss
anomaly-flagged companies

synonyms · verified questions

exasol-labs/exasol-semantic-views 0.5.1

Web app screens
dashboards per role, the same numbers

Ask Credora
Claude answers from the semantic layer first

Any SQL client
an analyst's GROUP BY gets the same rule

One semantic statement, as the RM

exasol-labs/preprocessor-framework 0.9.1

```
SELECT industry, approval_rate
FROM CREDORA_SEM.PORTFOLIO
```

→ PREPROC_RT.MASTER — owns the one system slot

→ **Rule for R_RM** — one rule per role, fail-open

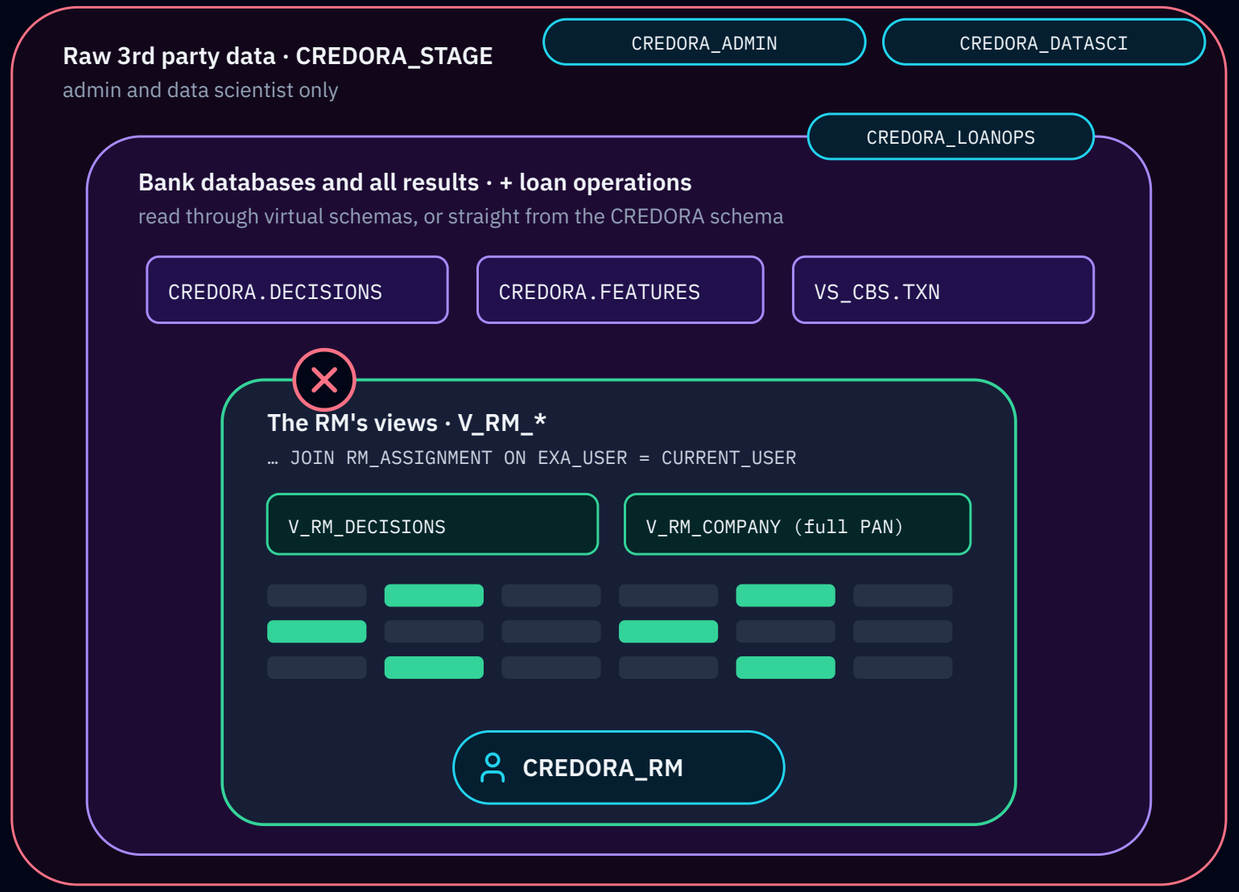
→ SEMANTIC_VIEWS_V1 — semantic-views 0.5.1 compiler

→ **Plain SQL over** V_LATEST_* (granted per role)

About 4 ms per statement for routed roles. CREDORA_SVC, which runs the jobs, has no rule and pays nothing.

Exasol, not the app, decides who sees what.

Four people, four Exasol users. A relationship manager reads only the V_RM_* views, which return the companies assigned to CURRENT_USER.



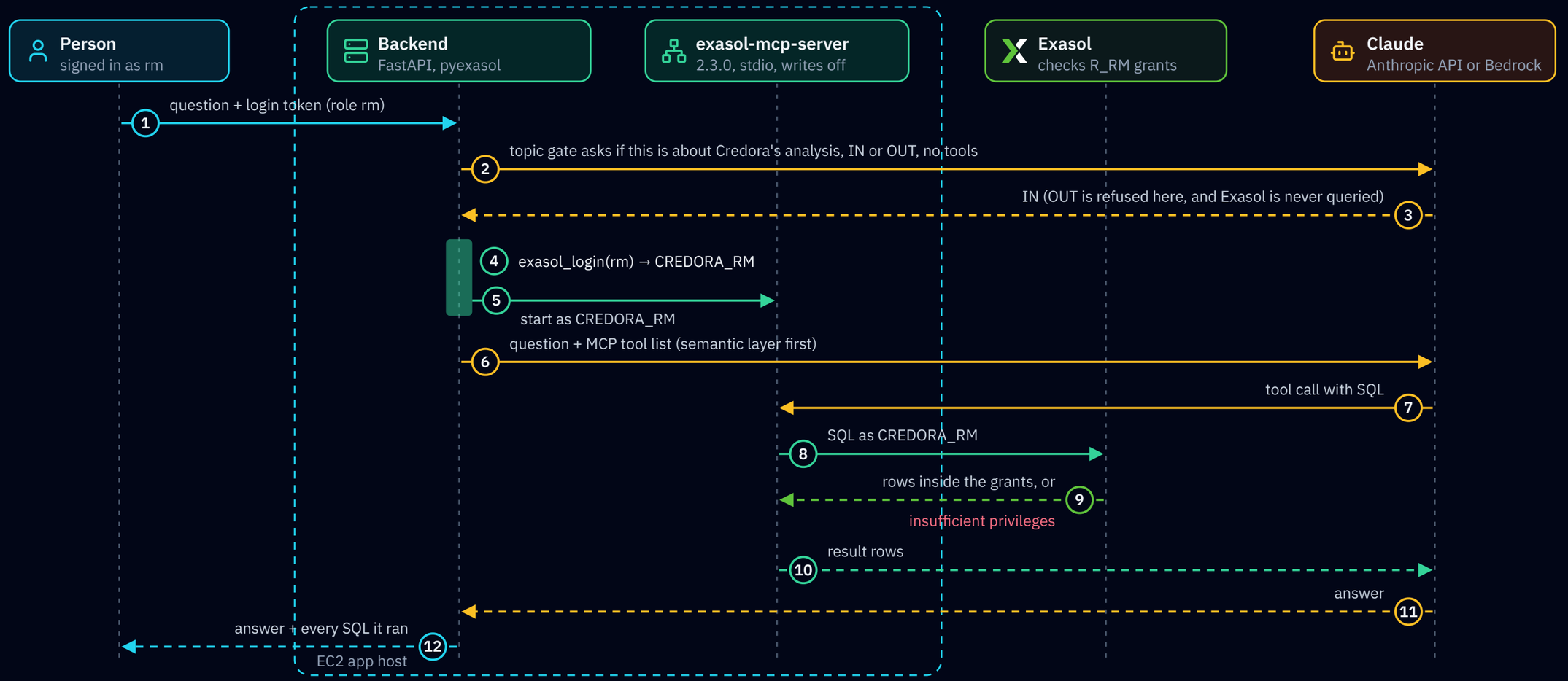

```
-- signed in to Exasol as CREDORA_RM (role R_RM)
CREDORA_RM> SELECT * FROM CREDORA.DECISIONS;
insufficient privileges
CREDORA_RM> SELECT * FROM CREDORA.V_RM_DECISIONS;
25 rows, the companies assigned to
CREDORA_RM, from published runs
```

```
CREATE VIEW CREDORA.V_RM_DECISIONS AS
SELECT d.* FROM CREDORA.DECISIONS d
JOIN CREDORA.RM_ASSIGNMENT a
  ON a.CIF_ID = d.CIF_ID
 AND a.EXA_USER = CURRENT_USER
JOIN CREDORA.RUN_PUBLISH p
  ON p.RUN_ID = d.RUN_ID;
```

The view runs with its owner's rights, so R_RM needs no grant on the tables underneath. In production each RM gets an own Exasol user, and the same views work unchanged.

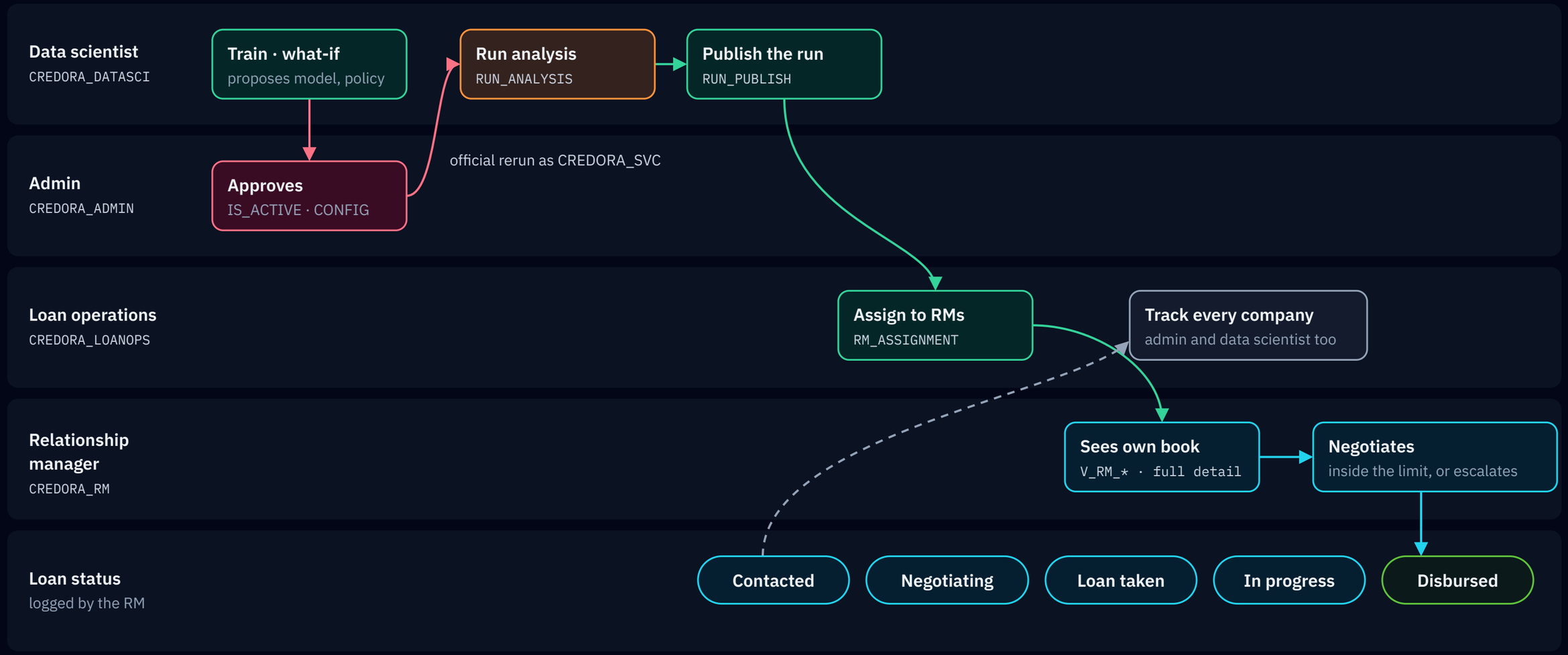
Ask Credora logs in as you, so the AI cannot read more than you.

Claude reaches Exasol only through the official Exasol MCP server (exasol/mcp-server), started for each question on the app host as the signed-in role's own Exasol user.

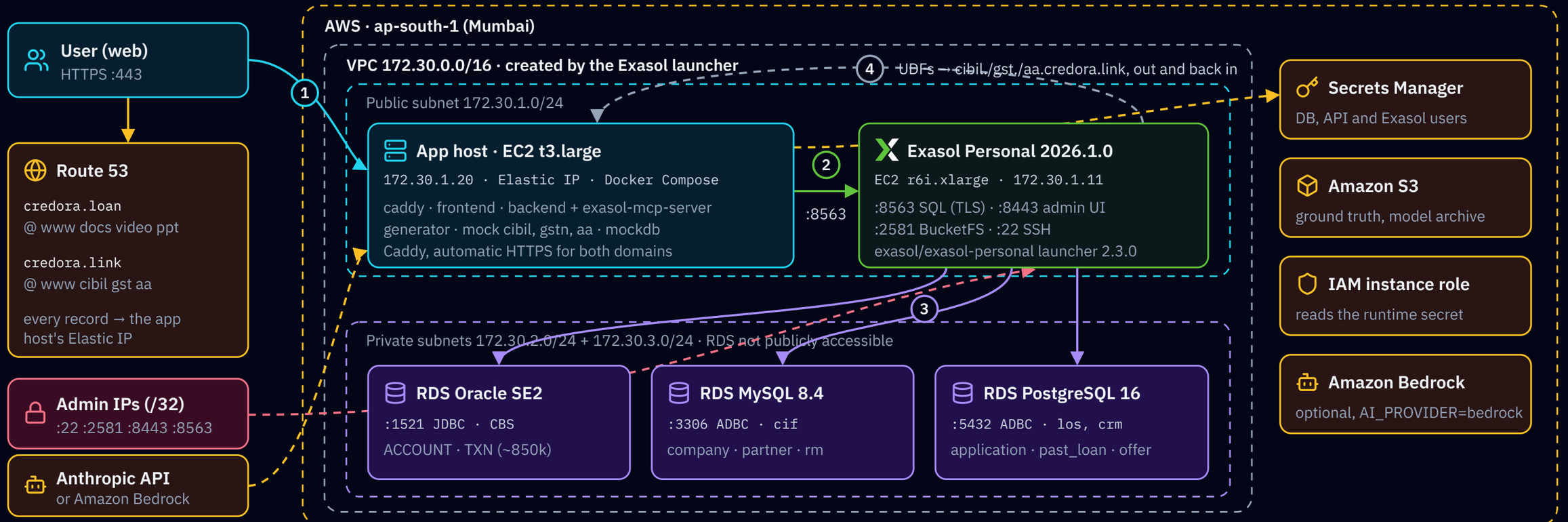


From a published run to a disbursed loan, in four roles.

Each hand-off is a row in Exasol or the CRM, so admin, data scientist and loan operations can track every company.



Started on the starter kit, deployed to AWS with one command.



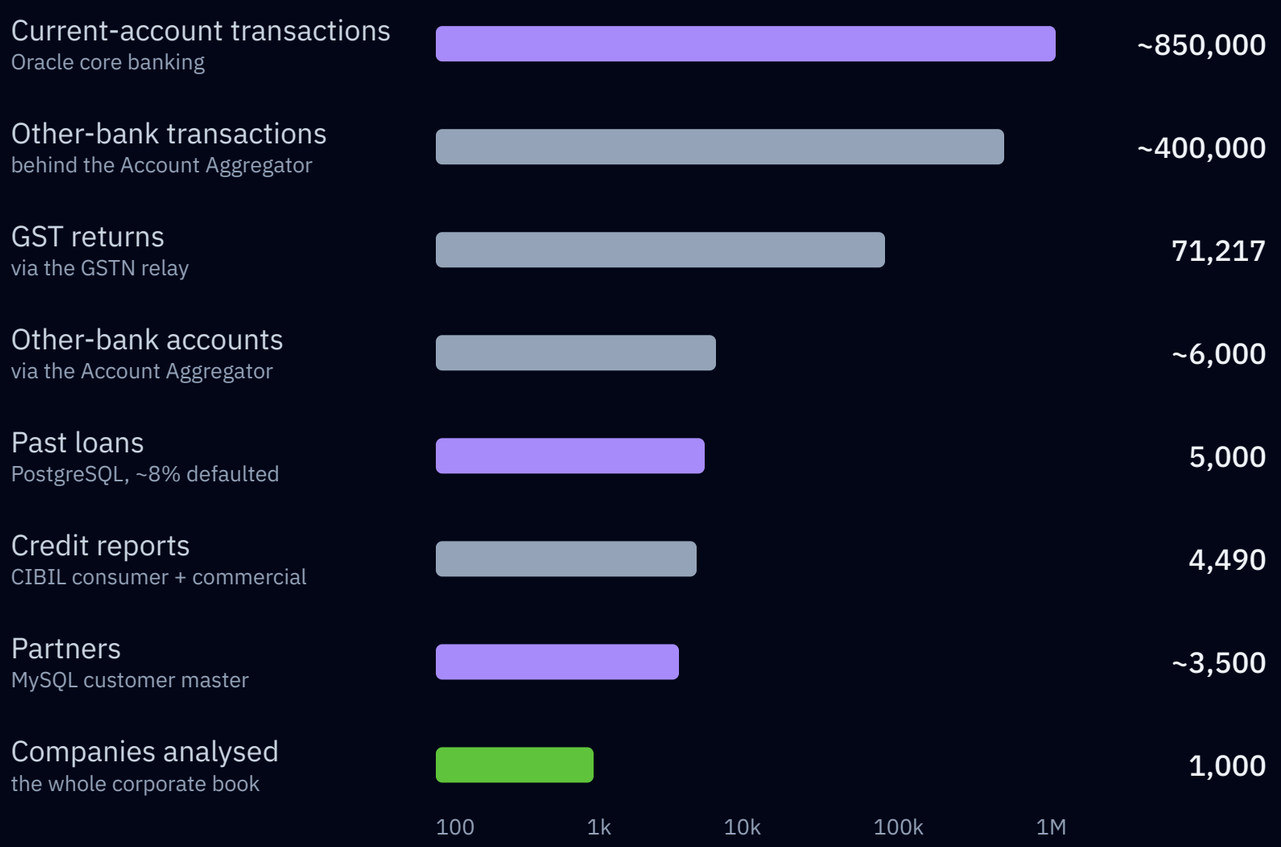
Before AWS, on a laptop
Exasol Personal Local Starter Kit (exakit 0.2.3) with local Exasol, MCP setup, exapump, pyexasol

```
$ python deploy/credora.py deploy --env <name>
```

Exasol on EC2, launcher 2.3.0	RDS ×3, app host, S3, DNS (OpenTofu)	App, mocks, sites (Compose)	1,000 synthetic companies	SLC, adapters, UDFs, Lua, roles, semantics	First training and analysis

Realistic volumes for a mid-size corporate book.

The data is synthetic, but the volumes are not small. Rows per source on a log scale, and what one run costs.



17.5s
the full analysis of 1,000 companies, 13 steps, one Exasol call

0.756
AUC of the champion. The challenger scorecard reaches 0.769

16
UDF instances fetching 3rd party data via API in parallel

30
companies flagged by the Isolation Forest, and 4 approvals became referrals

~75s
to regenerate all the synthetic data, with 1,000 companies, ~3,500 partners, every source rebuilt consistently. Semantic routing adds ~4 ms per statement.

Narrow doors, and failures that stay contained.

Network
Only 80 and 443 are public. Exasol's ports open to admin IPs only, and RDS sits in private subnets.

Database grants
A user per role. A refused query reaches the app as `403 Exasol denied access for your role`.

AI in the same boundary
MCP writes off, a topic gate before any query, every SQL statement shown with the answer.

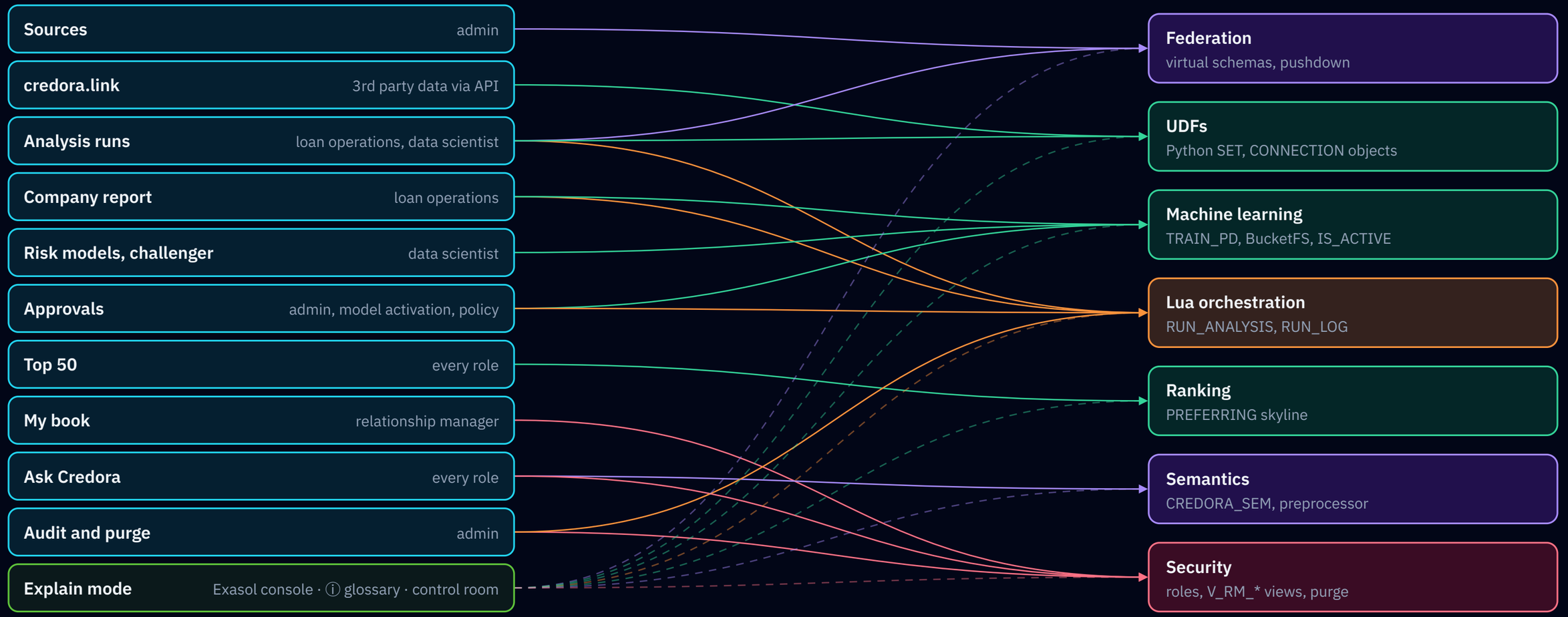
Secrets
Generated per deployment into Secrets Manager. API tokens live in CONNECTION objects. None in the repo.

Consent and retention
3rd party replies carry a purge time. PURGE_EXPIRED deletes them and keeps a SHA-256 proof.

If this fails	What happens	Recovery
A bank database	That step fails and is logged, and the last published run stays visible	Re-run when it is back
A 3rd party API	The fetch fails with the API's error, and nothing half-done is published	Re-run, the relays are stateless
A preprocessor module	The framework skips it (fail-open), and plain SQL keeps working	Fix or remove its rule
The Exasol node	The app shows the last results it can reach, and analysis and AI pause	Restart, and `deploy` recreates objects
The app host	The site is down, but Exasol and the databases are unaffected	`credora.py ship`
The MCP server	Ask Credora returns 503, and the rest of the app works	Ask again for a fresh process
A question outside grants	Exasol refuses, and Claude says the role cannot see it	None, this is access control working

Every screen in the demo proves an Exasol feature.

Explain mode shows it live, with an Exasol console showing the SQL behind each screen, an ⓘ glossary, and a control room.



Built with Exasol open source. Thank you.

Every project below runs in the live deployment, from both GitHub organisations, exasol and exasol-labs. Thank you to the Exasol team and the contributors.

exasol-labs/exasol-personal-local-starterkit
The starter kit (exakit 0.2.3) Credora began on, with local Exasol, MCP setup, exapump and pyexasol

exasol/exasol-personal
Exasol Personal and its launcher (2.3.0, Exasol 2026.1.0), the database itself, deployed on AWS with one command

exasol/oracle-virtual-schema
VS_CBS reads core banking in RDS Oracle live, with pushdown

exasol-labs/adbc-vs-rs
VS_CIF and VS_LOS read the MySQL customer master and PostgreSQL loan origination live through ADBC

exasol-labs/language-container-rs
The Rust script language container that runs the adbc-vs-rs adapter

exasol/script-languages-release
The Python 3 and Java containers behind the API, ML and AI UDFs

exasol-labs/exasol-semantic-views
CREDORA_SEM.PORTFOLIO, certified metrics shared by the app and the AI

exasol-labs/preprocessor-framework
Owns the SQL preprocessor slot and routes semantic SQL per role

exasol/mcp-server
exasol-mcp-server 2.3.0. Ask Credora's AI queries Exasol through it, as the signed-in role's user

exasol/pyexasol
The Python driver used by the backend and the deploy tool

exasol-labs/exapump
SQL and bulk-load CLI from the starter kit, used while building

exasol/virtual-schemas
The virtual schema framework the Oracle adapter builds on

Credora
for Sapphire Bank

Contributions welcome. Want to build on Credora, adapt it, or buy or license it?

Raise an issue at **github.com/anamika-iit-kgp/credora**. Credora is an ExaThon India 2026 project, a hands-on try-out of the Exasol Personal Local Starter Kit and Exasol Personal, deployed on AWS.

Raise an issue
github.com/anamika-iit-kgp/credora

credora.loan
the app · one-click sign-in for all four roles

docs.credora.loan
architecture, diagrams, why Exasol

credora.link
3rd party data via API. CIBIL, GSTN, Account Aggregator

Built for ExaThon India 2026 · Powered by Exasol Personal

Project by Anamika, Product Manager @ Equitas Small Finance Bank. Sapphire Bank is fictional and all data is synthetic.